MonoPod v0.8.4 — архитектура USB/iTunes и работа с медиатекой

Дата: 28.10.2026
Целевая плата: ESP32-S3 N16R8 (16 МБ Flash / 8 МБ PSRAM, проверить маркировку), два USB-C, ST7735S, microSD SPI, PCM5102A, TDA1308, зарядка TP4056, пьезоэлемент без вибрации.
Статус: исследование интерфейса iPod + реализация MonoPod USB Sync v2 + отдельный эксперимент с USB MSC. **iTunes-совместимость не подтверждена и пока не реализована.**

1. Что именно эмулируем

Исторические названия устройств различаются:

Поколение	Год	Модель	USB-синхронизация	Комментарий
iPod 2G	2002	A1019	Нет	Только FireWire; буквальный USB-профиль 2G не имеет исторического аналога
iPod 3G	2003	A1040	Да, с обновлением ПО	Цель исследования USB-совместимости
iPod 4G	2004	A1059	Да	Ближе к дизайну MonoPod с Click Wheel
iPod classic 6G	2007	A1238	Да	iTunesDB с hash58; гораздо сложнее

Выбор: профиль совместимости iPod 3G (USB, без hash58), а дизайн MonoPod остается похожим на 4G. Предлагать устройство как настоящий «iPod 2G по USB» технически противоречиво.

Источники: https://support.apple.com/en-ie/103823 ; https://www.apple.com/newsroom/2003/04/28Apple-Introduces-New-iPods/ ; https://github.com/strawberrymusicplayer/strawberry-libgpod/blob/master/README.over
.

2. Что было исследовано, а что нет

Исследованы опубликованные спецификации Apple, USB API Espressif, примеры `SysInfo`, описания `iTunesDB`, исходники libgpod, а также имеющаяся прошивка MonoPod v0.8.3. **Оригинальный бинарный образ прошивки iPod 3G не передавался и не дизассемблировался.** Это анализ протокола и файловой модели, а не заявление о полном реверсе ROM Apple.

Цепочка синхронизации

```
iTunes (Windows/macOS) → USB-определение iPod → USB Mass Storage (MSC/SCSI)
                                        ↓
                              microSD с FAT32
                                        ↓
                /iPod_Control/Device/SysInfo
                /iPod_Control/iTunes/iTunesDB
                /iPod_Control/Music/F00..Fxx/*.mp3|*.m4a
```

```
                                    ↓
               Safe Eject / USB disconnect
                                    ↓
          MonoPod монтирует microSD заново, читает iTunesDB
                                    ↓
                аудиодекодер → I²S → PCM5102A
```

Важное: USB CDC Serial v0.8.3 не является ни iAP1, ни USB MSC, и iTunes его как iPod не обнаружит. Для iPod 3G необходима как минимум эмуляция USB-накопителя, совместимые идентификационные данные и корректная работа с iTunesDB. Одного VID/PID и строки Serial недостаточно.

iTunesDB

- `mhbd` — заголовок базы.
- `mhsd` — набор данных, в том числе треки/плейлисты.
- `mhlt` — список треков.
- `mhit` — запись трека (ID, размер, тип, длительность, DBID).
- `mhod` — текстовые объекты: заголовок, артист, альбом, путь, жанр.
- Идентификатор трека из iTunes (`track_id / dbid`) должен оставаться стабильным. **Сравнение лишь по имени файла недопустимо** для полноценной синхронизации: повторное изменение тегов не должно создавать второй экземпляр.
- Для будущего импорта предусмотрен read-only тестовый анализатор `tools/itunesdb_inspect.py`; он не является полнофункциональным парсером всех версий и не пишет в базу.

3. Уникальный идентификатор ESP32-S3

Чип содержит заводской уникальный 48-битный MAC в eFuse. Доступ: `esp_efuse_mac_get_default(mac)` из `<esp_mac.h>`; интерфейс Wi-Fi включать не требуется. Это аппаратный идентификатор устройства, **не** заводской серийный номер Apple.

В прошивку добавлен `IpodIdentity.cpp`:

1. Читает 6 байт заводского MAC из eFuse.
2. Формирует постоянный 16-символьный hex GUID: `4D50` (метка MonoPod) + `AABBCCDDEEFF` (48 бит MAC).
3. При недоступности eFuse генерирует GUID с помощью `esp_random()` **один раз** и сохраняет в `Preferences/NVS` (`monopod-id/guid64`). В случае стирания NVS fallback может измениться. Нормальная прошивка не зависит от случайной генерации.
4. Возвращает `GUID=` в `MPOD1 INFO`. Самостоятельный MSC-скетч назначает этот идентификатор через `USB.serialNumber()`.

У настоящих iPod существуют разные поля `pszSerialNumber` и `FirewireGuid`. Для совместимости с iTunes может потребоваться соответствие модели, строки Serial, SCSI identity и содержимого SysInfo. Автоматическое принятие сгенерированного номера iTunes **не гарантировано**.

4. Фактическая вместимость microSD

Правильная величина определяется вставленной картой, а **не** 16 МБ Flash ESP32.

- В USB CDC Sync v2 запрос `INFO` возвращает SD=, TRACKS=, `TOTAL_BYTES=`, FREE_BYTES= и `GUID=` — по `SD.totalBytes()` и `SD.usedBytes()`.
- Нет карты: `SD=0`, объем = 0, команды PUT/REPLACE/DELETE отказывают с `ERR NO_SD`, на дисплее всплывает `No microSD card`.
- В USB MSC лабораторном режиме число блоков берётся из `SdCard::sectorCount()`, размер блока — 512 байт; в `USBMSC::begin` передается именно реальная геометрия. При отсутствии карты вызывается `MSC.mediaPresent(false)`, то есть SCSI «носитель отсутствует», а не фиктивный накопитель с бесплатным местом.

- Контроллер S3 USB-OTG — **Full Speed 12 Мбит/с**, не High Speed 480 Мбит/с. Фактическая запись будет медленнее теоретического максимума ~1,5 МБ/с.

Ограничение: ранние iPod, iTunes и FAT32 могут иметь особенности работы с картами 64/128/256 ГБ. Первые тесты выполнять с отдельной резервной FAT32 картой на 8–32 ГБ.

5. USB MSC: лабораторная реализация

Файл `examples/S3_USB_MSC_Lab/S3_USB_MSC_Lab.ino`:

- Для ESP32-S3; Arduino-ESP32 3.x + библиотека SdFat v2.
- Подключение microSD: SCK=GPIO12, MISO=GPIO13, MOSI=GPIO11, CS=GPIO14. GPIO10 (TFT CS) удерживается в HIGH.
- Arduino IDE: `ESP32S3 Dev Module`, `USB Mode = USB-OTG (TinyUSB)`, `USB CDC On Boot = Disabled`.
- Подключать **нативный** порт USB-C, соединенный с GPIO19/GPIO20, а второй порт использовать для прошивки/диагностики, если он работает через UART-мост. Проверить разводку конкретной платы.
- По умолчанию `LAB_ENABLE_WRITES=0`: безопасная проверка чтения и реального размера. `LAB_APPLE_DESCRIPTOR_TEST=0` — собственное название устройства.
- После резервного копирования экспериментально включить `LAB_ENABLE_WRITES=1`. **Нельзя одновременно монтировать FAT на ESP32 и предоставлять этот же носитель ОС для записи** — это повреждает файловую систему. Скрипт MSC не обращается к файлам после запуска USB.
- Для исследовательского сопоставления USB-идентификаторов предусмотрен флаг `LAB_APPLE_DESCRIPTOR_TEST=1` (0x05AC/0x1201, «iPod 3G»), **только как лабораторный тест**. Apple USB VID ей принадлежит; такое изменение не подтверждает совместимость, может вызвать неверную загрузку драйвера и не подходит для распространения устройства под чужим брендом.

Это отдельно прошиваемый стенд, а не готовая функция основного плеера. Для настоящей синхронизации надо перенести USB MSC в основную прошивку как эксклюзивное состояние: остановить воспроизведение, закрыть FAT, передать карту USB-хосту, после безопасного извлечения перемонтировать FAT и обновить библиотеку.

6. Что реализовано в MonoPod USB Sync v2

Это **рабочая на уровне исходников** собственная протокольная реализация (не iTunes):

Команда	Действие
`MPOD1 HELLO`	Протокол v2, наличие SD
`MPOD1 INFO`	Реальный объем, свободное место, GUID, треки
`MPOD1 PUT <size> <crc32> <filename>`	Загрузка нового файла (старый не перезаписывается)
`MPOD1 REPLACE <size> <crc32> <filename>`	Обновление существующего файла после проверки CRC
`MPOD1 DELETE <filename>`	Удаление файла только из `/Music`
`MPOD1 SCAN`	Перестроение индекса после изменений

`REPLACE` сначала пишет `/Music/.monopod-upload.part`; затем проверяет CRC32, записывает journal цели, перемещает старый файл во временную резервную копию, заменяет файл и чистит backup. После потери питания следующая загрузка попытается восстановить старый файл (если новая версия не была завершена). Свободного места должно хватить на **новую** копию + служебный запас. Это не полноценная журналируемая ФС — испытания с внезапным обрывом питания на реальной SD ещё обязательны.

`MonoPod Studio` теперь умеет **Upload / Update USB-C** с `replace=True` и **Delete USB track** с подтверждением. На самом MonoPod: в списке Songs/Category удержание Select открывает экран подтверждения удаления локального файла из `/Music`.

Важное ограничение: идентификация одинакового трека по одному имени файла пока поддерживается только в собственном `MPOD1`; для iTunes обязателен учет `track_id/dbid` из `iTunesDB`. Файлы внутри `/iPod_Control/Music` нельзя удалять вручную без изменения базы.

7. AAC, M4A и ALAC

Формат	Сейчас, v0.8.4	Реализуемость на ESP32-S3
MP3	Да, через ESP8266Audio	Да
WAV PCM	Да	Да
FLAC	Да при FEATURE_FLAC	Да
AAC с ADTS, `.aac`	Да при FEATURE_AAC	Да
AAC-LC в контейнере `.m4a`	**Нет** (нужен MP4/M4A demuxer)	Да; Espressif ESP Audio Codec поддерживает M4A
ALAC в контейнере `.m4a`	**Нет** (нужен MP4 demux + ALAC decoder)	Да в принципе, производительность проверить
DRM-защищенный AAC	Нет	Отдельный вопрос лицензий/DRM, не заявляется

Официальные Espressif `esp_audio_codec` и `esp-adf-libs` поддерживают AAC и ALAC; для AAC-LC 48kHz stereo указан ориентир ~6.75% CPU на S3R8 в определенном тесте (не измерение MonoPod). Для ALAC библиотека доступна, но декодирование 44.1 кГц/16 бит и 96 кГц/24 бита нужно отдельно замерять. PCM5102A поддерживает воспроизведение 16-битных PCM потоков после преобразования из ALAC; 24-битный путь требует отдельной проверки выбранной I^2S реализации и DSP.

Реализация будущего декодера: `M4A demux (moov/mdat, stsz/stco/stsc, alac config) → AAC/ALAC decoder → interleaved PCM → I²S output`. Следует отключать недоступный декодер, а не обещать воспроизведение только потому, что файл принят при синхронизации.

Источники: https://github.com/espressif/esp-adf-libs/blob/master/esp_audio_codec/README.md ; https://docs.espressif.com/projects/esp-adf/en/latest/basic-components/audio-codec/esp-audio-codec.html ; https://github.com/earlephilhower/ESP8266Audio .

8. План достижения реальной совместимости с iTunes

1. **USB identity** — USBPcap/Wireshark захват логов оригинального iPod 3G либо сверенные описания MSC/SCSI inquiry + дескрипторов. Подбирать VID/PID без знания запросов — недостаточно.
2. **MSC (эксклюзивный)** — перенести лабораторный read-only прототип в основную прошивку; удостовериться, что FAT не используется двумя сторонами одновременно; затем осторожно разрешить запись.
3. **SysInfo** — формировать совместимые ключи модели/серийника/FirewireGuid из eFuse, сохранить согласованность с USB ID.
4. **iTunesDB** — добавить полноценный парсер с проверкой границ, обработкой `mhbd/mhsd/mhlt/mhit/mho` треков AAC и плейлистов; сравнить `track_id/dbid` до и после синхронизации.
5. **Повторная синхронизация** — после измененных метаданных iTunes обновляет исходную запись вместо создания новой; файлы удалённых треков и плейлисты должны быть согласованы с базой.
6. **No SD** — MSC `medium not present`, предупреждение на дисплее, нет ложного «пустого диска», загрузка заблокирована.

7. **AAC/ALAC** — портировать демультиплексор M4A и декодеры, затем измерить heap/PSRAM, пропуски I²S, CPU и ток.
8. **Испытания** — Windows 10/11 iTunes 12.x, Mac (если применимо), карты FAT32 8/32/128 ГБ, повторная синхронизация, USB извлечение, обрыв питания, полная библиотека >1024 треков. На отдельной тестовой microSD до прохождения протокола.

9. Проверка нынешней версии

В Arduino IDE для основного MonoPod **USB CDC On Boot = Enabled** (режим v2, порт MPOD1). Для MSC-лаборатории **USB Mode = USB-OTG/TinyUSB**, `USB CDC On Boot = Disabled`; режимы несовместимы в представленном виде и требуют перепрошивки при переключении.

```
python -m pip install -r Studio/requirements.txt
python Studio/usb_sync.py ports
python Studio/usb_sync.py --port COM7 info
python Studio/usb_sync.py --port COM7 send "C:/Music/song.mp3"
python Studio/usb_sync.py --port COM7 --replace send "C:/Music/song.mp3"
python Studio/usb_sync.py --port COM7 delete song.mp3
python tools/itunesdb_inspect.py path/to/iPod_Control/iTunes/iTunesDB --limit 30
python -m pytest -q tests
```

Тесты Python являются **симуляцией протокола и статическими проверками**. Полная сборка Arduino-ESP32, USB enumeration, распознавание iTunes и реальное AAC/ALAC воспроизведение пока **не подтверждены**.

10. Сводка статуса

Задача	Статус
Заводской уникальный ID ESP32-S3	Реализовано чтение из eFuse, fallback NVS
Реальный объем и свободное место microSD в MPOD1	Добавлено
Нет карты → запрет загрузки и предупреждение на TFT	Добавлено в код
Замена существующего файла без дубля с тем же именем	Добавлено в MPOD1 и Studio
Удаление трека из Studio и на плеере	Добавлено для `/Music`
Реальный USB MSC накопитель	Отдельный лабораторный скетч, не основной плеер
iTunes распознает MonoPod как iPod 3G	**Не реализовано, не проверено**
Парсер `iTunesDB` на ESP32	**Не реализован**; есть read-only исследовательский Python-инспектор
AAC `.aac` ADTS	Есть в текущем движке
AAC `.m4a` / ALAC `.m4a`	**Не реализовано**, технически возможно

Ключевые источники

- Apple: https://www.apple.com/newsroom/2003/04/28Apple-Introduces-New-iPods/
- Apple: https://support.apple.com/en-ie/103823
- Libgpod format and hash variants: https://github.com/strawberrymusicplayer/strawberry-libgpod/blob/master/README.overview
- Example SysInfo: https://github.com/dstaley/ipod-sysinfo
- Official ESP32-S3 eFuse MAC: https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/misc_system_api.html
- Official Arduino USB MSC: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/usb_msc.html
- Official USB descriptors: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/usb.html
- ESP32-S3 USB speed: https://docs.espressif.com/projects/esp-iot-solution/en/latest/usb/usb_overview/usb_o
- Espressif codec/M4A: https://github.com/espressif/esp-adf-libs/blob/master/esp_audio_codec/README.md